Zen AI Model Family

Zen-Agent

Autonomous Task Execution with Long-Horizon Planning

Technical Report v2025.01

Antje Worring, Zach Kelling Hanzo AI Research Team* Zoo Labs Foundation†

January 2025

Abstract

We present **Zen-Agent**, a 32-billion parameter model trained for autonomous task execution, multi-step planning, and compositional tool use. Zen-Agent is built on the Zen MoDE (Mixture of Distilled Experts) architecture and fine-tuned on the Zen Agentic Trajectory Dataset—a corpus of 12.4 million complete task trajectories spanning software engineering, web navigation, data analysis, and scientific reasoning. The model achieves 71.2% on GAIA (Level 1–3 average), 42.8% on SWE-bench Verified, and 88.3% on HumanEval, establishing a new frontier for open agentic models at this parameter scale. Zen-Agent introduces **Hierarchical Task Decomposition (HTD)**: an explicit planning module that generates subgoal trees before executing any tool calls, substantially reducing failure modes caused by premature commitment to incorrect action sequences. The model natively speaks the Model Context Protocol (MCP), enabling zero-configuration integration with the Hanzo MCP tool ecosystem of 260+ tools.

Contents

1 Introduction **3**
1.1 Model Summary . . . 3

2 Architecture **3**
2.1 Zen MoDE 32B Backbone . . . 3
2.2 Hierarchical Task Decomposition (HTD) . . . 4
2.3 Goal Persistence Module . . . 4
2.4 MCP-Native Tool Interface . . . 5

3 Training **5**
3.1 Zen Agentic Trajectory Dataset . . . 5
3.2 Training Protocol . . . 5

4 Evaluation **6**
4.1 GAIA Benchmark . . . 6
4.2 SWE-bench Verified . . . 6
4.3 HumanEval . . . 6
4.4 Tool Use Accuracy . . . 7
4.5 Trajectory Efficiency . . . 7

*research@hanzo.ai
†foundation@zoo.ngo

5 Applications **7**
5.1 Software Engineering Automation . . . 7
5.2 Research Automation . . . 7
5.3 IT Operations . . . 7

6 Safety and Constraints **7**
6.1 Action Reversibility Classification . . . 7
6.2 Budget Limits . . . 8
6.3 Hallucination Reduction . . . 8

7 Integration **8**

8 Related Work **8**

9 Conclusion **8**

1 Introduction

The transition from conversational AI to agentic AI—systems that take sequences of actions in the world to complete long-horizon tasks—demands a qualitative shift in how models are trained and evaluated. A conversational model optimizes a single response; an agentic model must plan, execute, observe feedback, revise beliefs, and persist toward a goal across potentially hundreds of tool-mediated steps.

Existing large language models, even those capable of impressive single-turn reasoning, fail on agentic benchmarks primarily due to:

1. **Premature commitment**: Taking tool calls before adequately planning the task structure, leading to irrecoverable state.
2. **Context drift**: Losing track of the original goal as the trajectory lengthens.
3. **Tool misuse**: Calling tools with syntactically correct but semantically wrong arguments, wasting steps and budget.
4. **Error propagation**: Failing to detect and recover from subtask failures before they cascade into total task failure.

Zen-Agent addresses each failure mode through architectural and training innovations described in this report.

1.1 Model Summary

Property	Value
Parameters	32B
Architecture	Zen MoDE 32B (8 experts, top-2)
Context Length	128K tokens
Max Trajectory Length	512 tool calls
Native Protocols	MCP, OpenAI Tools, Anthropic Tools
Supported Runtimes	Python, Node.js, Docker, Kubernetes
Training Trajectories	12.4M task trajectories

Table 1: Zen-Agent Model Specifications

2 Architecture

2.1 Zen MoDE 32B Backbone

Zen-Agent uses the Zen MoDE architecture at 32B scale: 40 transformer layers with 32 attention heads, grouped-query attention (GQA with 8 KV heads), and sparse MoE feed-forward networks. Each MoE layer maintains 8 experts with top-2 routing, yielding approximately 8B active parameters per forward pass while providing 32B parameter capacity for specialist knowledge.

Component	Specification
Layers	40
Attention heads	32 (GQA, 8 KV heads)
Hidden dimension	5120
MoE experts per layer	8 (top-2 routing)
Active params per token	≈8B of 32B total
Context length	128K (RoPE, $\theta = 500{,}000$)
Vocabulary	152,064 tokens

Table 2: Zen MoDE 32B Architecture

2.2 Hierarchical Task Decomposition (HTD)

Before any tool call is issued, Zen-Agent generates an explicit **task plan** in a structured thinking block. The plan takes the form of a tree of subgoals:

```
{
  "goal": "Fix the authentication bug in PR #1247",
  "subgoals": [
    {
      "id": "sg1",
      "description": "Reproduce the bug",
      "steps": ["checkout branch", "run failing tests", "read error"],
      "depends_on": []
    },
    {
      "id": "sg2",
      "description": "Identify root cause",
      "steps": ["read auth module", "trace call stack"],
      "depends_on": ["sg1"]
    },
    {
      "id": "sg3",
      "description": "Implement fix and verify",
      "steps": ["edit file", "run tests", "verify no regressions"],
      "depends_on": ["sg2"]
    }
  ]
}
```

Listing 1: HTD Plan Structure

The HTD module is implemented as a constrained decoding prefix: the model always generates a `<plan>` block before the first `<tool_call>` block. Training enforces this through a curriculum that penalizes trajectories where tool calls precede planning.

2.3 Goal Persistence Module

To combat context drift over long trajectories, the model maintains a **goal stack** in its working memory: a compact structured summary of the original task and completed subgoals. At every 32-step checkpoint, the model re-reads the goal stack and emits a brief `<goal_check>` block confirming alignment. Divergence from the goal triggers automatic backtracking to the last consistent state.

2.4 MCP-Native Tool Interface

Zen-Agent is the first model trained natively on the Model Context Protocol (MCP) tool call format. This means:

- Tool schemas are embedded in the model's training distribution, not injected at inference.
- Common MCP tools (bash, file I/O, web browser, code execution) are callable without schema prompting.
- The model understands MCP error codes and applies appropriate retry strategies.
- Tool results are parsed and integrated into the plan without additional prompting.

3 Training

3.1 Zen Agentic Trajectory Dataset

Domain	**Trajectories**	**Proportion**	**Source**
Software engineering	4,100,000	33.1%	GitHub issues, PRs
Web navigation	2,800,000	22.6%	Browser sessions
Data analysis	1,900,000	15.3%	Jupyter notebooks
Scientific reasoning	1,400,000	11.3%	Research papers
Coding tasks	1,200,000	9.7%	Competitive coding
System administration	1,000,000	8.1%	DevOps runbooks
Total	**12,400,000**	100%	

Table 3: Zen Agentic Trajectory Dataset Composition

Each trajectory record contains: task specification, tool call sequence, tool outputs, final answer, success/failure label, and a human-written critique. Trajectories with more than 10 consecutive failed tool calls were filtered out during dataset curation.

3.2 Training Protocol

Training proceeds in four stages:

Stage 1 – SFT on correct trajectories (50K steps): The model is fine-tuned on the 43% of trajectories labeled as fully successful, using standard next-token prediction loss over the complete trajectory (plan + tool calls + observations + answer).

Stage 2 – DPO on trajectory pairs (30K steps): For each task with both successful and failed trajectories, we construct preference pairs and apply Direct Preference Optimization to increase the probability of success-leading prefixes.

Stage 3 – GRPO for tool accuracy (20K steps): Group Relative Policy Optimization with a sparse reward signal: $+1$ for task success, -0.1 per unnecessary tool call (calls that could have been skipped given the information already available in context).

Stage 4 – Safety fine-tuning (10K steps): Constitutional AI prompts are used to discourage trajectories that involve irreversible destructive actions without explicit user confirmation steps.

Stage	Steps	Batch	LR
SFT	50K	64	1e-5
DPO	30K	32	5e-6
GRPO	20K	16	2e-6
Safety	10K	32	1e-6

Table 4: Zen-Agent Training Configuration

4 Evaluation

4.1 GAIA Benchmark

GAIA measures real-world assistant capability across general AI tasks requiring web search, file processing, code execution, and multi-step reasoning. Results below are on the official GAIA validation set (Level 1 = easy, Level 3 = hard).

Model	Level 1	Level 2	Level 3	Average
GPT-4o (2024-11)	73.2	52.1	28.4	51.2
Claude 3.5 Sonnet	77.3	56.2	31.7	55.1
Gemini 1.5 Pro	70.1	48.3	26.1	48.2
Open-source SoTA (32B)	52.7	31.4	14.2	32.8
Zen-Agent 32B	**83.1**	**68.4**	**42.3**	**71.2**

Table 5: GAIA Benchmark Results (%, higher is better)

4.2 SWE-bench Verified

SWE-bench Verified presents real GitHub issues from popular Python repositories; the model must produce a patch that passes the official test suite without seeing the tests.

Model	Resolve Rate (%)	Avg. Tool Calls
GPT-4o (Agentless)	33.2	24.3
Claude 3.5 Sonnet	38.1	31.7
SWE-agent + Claude	39.4	48.2
OpenHands + Claude	41.0	52.1
Zen-Agent 32B	**42.8**	**29.6**

Table 6: SWE-bench Verified Results

Zen-Agent achieves the highest resolve rate while requiring 30% fewer tool calls than the next-best method, demonstrating the efficiency gain from Hierarchical Task Decomposition.

4.3 HumanEval

Single-function code generation evaluated by executing generated code against hidden test cases.

Model	pass@1 (%)
GPT-4o	86.4
Claude 3.5 Haiku	81.2
Gemini 1.5 Flash	78.3
Zen-Agent 32B	**88.3**

Table 7: HumanEval pass@1

4.4 Tool Use Accuracy

We evaluate tool argument correctness on a held-out set of 10,000 tool calls annotated with ground-truth arguments. Zen-Agent achieves 91.4% exact-match accuracy on tool arguments, compared to 78.2% for the best-performing comparable baseline.

4.5 Trajectory Efficiency

Task Type	Success Rate	Avg Calls	Backtrack Rate
Code debugging	84.3%	18.2	12.1%
Web research	91.2%	12.7	6.3%
Data analysis	87.6%	22.4	9.8%
File manipulation	96.1%	7.3	2.1%
System administration	78.4%	31.2	18.7%

Table 8: Zen-Agent Trajectory Efficiency by Task Type

5 Applications

5.1 Software Engineering Automation

Zen-Agent powers the Hanzo Code service, which autonomously handles GitHub issues labeled `zen-agent`: the model reads the issue, explores the codebase, writes a patch, runs the test suite, and opens a pull request with a structured description of its changes. Average cycle time is under 3 minutes for issues categorized as medium complexity.

5.2 Research Automation

For scientific research tasks, Zen-Agent can execute multi-hour workflows involving literature search (via web tools), data download and preprocessing, statistical analysis (via Python execution), and report generation—all from a single natural language task specification.

5.3 IT Operations

Zen-Agent is deployed in the Hanzo cloud platform for automated incident response: given an alert, the model diagnoses the root cause by querying logs, metrics, and configuration state, then executes a remediation playbook or escalates with a structured diagnostic report.

6 Safety and Constraints

6.1 Action Reversibility Classification

Every tool in the Zen MCP ecosystem is annotated with a reversibility label: `reversible`, `soft-destructive` (data can be recovered), or `hard-destructive` (action is irreversible). Zen-

Agent refuses to execute `hard-destructive` actions without an explicit user confirmation step in the trajectory.

6.2 Budget Limits

Each agent session is configured with maximum tool calls (default 200), maximum wall-clock time (default 10 minutes), and maximum cost per session (configurable, default $0.50). The model monitors its own budget and gracefully summarizes progress when approaching limits.

6.3 Hallucination Reduction

Zen-Agent's training emphasized grounded claims: the model is penalized in GRPO training for asserting facts about tool outputs that contradict what was actually returned. On the TruthfulQA benchmark, Zen-Agent achieves 81.3% accuracy, 4.2 points above the backbone pre-fine-tuning baseline.

7 Integration

```
from zen import ZenAgent

agent = ZenAgent(
    model="zenlm/zen-agent-32b",
    tools=["bash", "file", "web", "python"],
    max_steps=100,
    budget_usd=1.0,
)

result = agent.run(
    "Analyze the performance regression in the /api/search endpoint "
    "using the last 24h of production logs and propose a fix.",
    context={"log_path": "/var/log/api/search.log"}
)

print(result.answer)
print(f"Steps taken: {result.num_steps}")
print(f"Cost: ${result.cost_usd:.4f}")
```

Listing 2: Zen-Agent Task Execution

8 Related Work

ReAct [1] established the interleaved reasoning-action paradigm. Tree-of-Thought [2] introduced explicit planning over branching action spaces. SWE-agent [3] specialized for software engineering via shell-based interaction. Zen-Agent advances this line through HTD, native MCP integration, and large-scale trajectory training at 32B scale with the Zen MoDE architecture.

9 Conclusion

Zen-Agent demonstrates that agentic capability can be substantially improved by training on real task trajectories at scale, enforcing explicit hierarchical planning before action execution, and natively integrating with tool ecosystems via MCP. The resulting model achieves 71.2% GAIA and 42.8% SWE-bench Verified, establishing a new state of the art for open agentic models at 32B scale while using 30% fewer tool calls than comparable methods.

Future work will extend Zen-Agent to multi-agent collaboration (where multiple Zen-Agent instances coordinate via MCP), longer planning horizons (beyond 512 tool calls), and continual learning from deployment feedback via the Zoo DSO protocol.

References

[1] S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," ICLR, 2023.

[2] S. Yao et al., "Tree of Thoughts: Deliberate Problem Solving with Large Language Models," NeurIPS, 2023.

[3] J. Yang et al., "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering," NeurIPS, 2024.